A multi-model relationship detection method to assist with naïve exploration of high-dimensional data

James C. Wiley, Simon G. English, Kinsey Church, Richard A. Ward, James Flowers, Céline Chamoun

Released February 2025

Licence: Creative Commons Attribution 4.0 International License

Contact email – jamescwiley@outlook.com

Abstract

This paper presents a method for classifying predictors X_p based on their relationship to an outcome variable Y; as having main effects, interactions, collinearity, or no effects. The presented method operates by combining complimentary information from a multivariate model and a series of bivariate models. We demonstrate how the method works using simulated data. In addition, we experimentally vary the effect sizes in our data generation process to see if the proposed method can detect different relationships between predictors X_p and outcome Y at varied strengths. We also vary the sample size (n) and observe the impact on relationship classification. We find that the proposed method functions as desired within the constraints of this study. We propose future simulation designs for continued testing of said method. We conclude by providing broad instructions for applying this method. Our goal is to use this method to develop initial analytical profiles of high-dimensional data in naïve data exploration contexts. This work stems from trying to find an efficient alternative to scatterplot matrices when exploring data that contain thousands of variables.

Keywords: data exploration, interaction detection, relationship classification, relationship detection, scatterplot matrix

Introduction

This paper presents a method designed to classify predictors X_p into four categories based on their multivariate relation to an outcome variable Y: distinctly important; contains an interaction; is collinear with other variables; and is unimportant. To our knowledge, this is the first paper to present a method that combines complimentary information from a series of models to classify multivariate relationships. Our intention is to find a method that can provide similar insights on variable relationships as scatterplot matrices (Cleveland, 1993; Kutner et al., 2005), but for datasets with many variables. Scatterplot matrices involve building a bivariate scatterplot for each possible variable-pair (e.g., see Figure 1). This is useful when working with smaller datasets, but examining every variable-pair becomes tedious when working with thousands of variables (i.e., high-dimensional data; Strobl et al. 2008). Although the present method does not provide as much information as would be gained by examining a scatterplot matrix, it does provide some overlapping information, whilst being substantially less tedious to analyse.

The multi-model method

We will first give a general overview of our approach. This overview will refer to multivariate and bivariate models generally. Later, we discuss which specific modelling techniques we prefer to use—though analysts can use any multivariate and bivariate modelling methods they like, so long as they are cognisant of the limitations of those modelling methods.

Given a specific outcome variable *Y* to model, the approach in question combines:

- a multivariate model fitted using all potential predictors (X_1, X_2, X_3, … X_p; where p = the number of predictors);
- with bivariate models examining the direct relationship between each predictor X_p and the outcome *Y*.

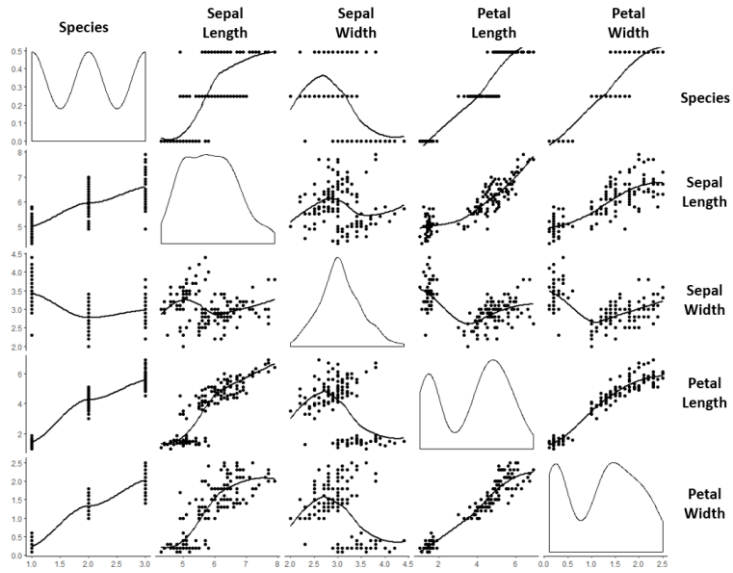

Figure 1. Scatterplot matrix of Fisher's (1936) iris dataset. Created using the GGally library (Schloerke et al., 2018) in R version 3.4.1 (R Core Team, 2017).

We assign two importance scores to each predictor, one score showcasing the importance of the predictor in the multivariate context and one showcasing the importance of the predictor in the bivariate context. These importance scores can be any metric specific to the modelling method used, so long as they indicate the relative importance of each predictor X_p in the model. For example, below we will use the Garson (1991; Goh 1995) importance metric for neural networks, and variance accounted for (R-squared) from LOESS models (Kutner et al., 2005).

So long as the multivariate model authentically represents the underlying data generation process, it should rate variables that relate to the outcome highly, including those with interaction effects. Variables that are collinear with direct predictors are expected to have diluted importance scores. Variables with little to no relationship with the outcome variable should exhibit negligible importance scores.

In the bivariate model, we expect predictors that interact with other predictors to have lower importance scores relative to their multivariate importance scores, because the interaction information is not present. Additionally, variables that are collinear with strong predictors will showcase strong bivariate relationships with the outcome, thus having high bivariate importance scores. Finally, direct predictors and irrelevant variables should have high and low respective bivariate importance scores.

Taken together, we conclude that:

- a predictor with *higher multivariate* and *higher bivariate* importance scores is *distinctly important*;
- a predictor with *higher multivariate* and *lower bivariate* importance scores has an *interaction*;
- a predictor with *lower multivariate* and *higher bivariate* importance scores is *collinear* with another strong predictor; and
- a predictor with *lower multivariate* and *lower bivariate* importance scores is *unimportant*.

This information is visualized in tables 1.1 and 1.2, along with example output. For sake of simplicity, we discuss variable importance scores within a "high" versus "low" dichotomy, though in reality these metrics are continuous; potentially yielding medium importance variables and semi-ambiguous classifications. Results are formatted so that cells with higher importance scores are coloured a deeper shade of purple or blue (the former being used for the multivariate model; the latter being used for the bivariate model). We recommend relying heavily on these visual cues when trying to determine which variables cluster together within the same classifications.

Note that we judge whether a predictor has higher or lower importance relative to other predictors in the dataset. This highlights a distinct assumption of our method: that variables with each kind of relationship are prevalent within the data (i.e., high importance relations, interactions, collinear relations, low importance relations, and medium importance relations). If any relationship is missing, results may be muddled, because there will not be enough variation in the importance scores to spot any particular relationship relative to other relationships. For example, if no variables are strong predictors of the outcome, variation in importance scores will be arbitrary and thus meaningless. Alternatively, if all variables are highly collinear,

but together have high predictive power, they may all appear unimportant. We label this the *diverse context assumption* because we assume there are all types of predictor relationships in the dataset (e.g., direct, interaction, collinear, etc.).

In addition to the diverse context assumption, analysts should be aware of the assumptions and limitations of the particular models they choose to implement in place of either the multivariate or bivariate models.

Related work

Other authors have empirically tested whether particular models can detect interactions (Wright et al. 2016; Winham et al. 2012) and multicollinearity (Toloşi et al. 2011) in data exploration contexts. This work is typically situated within bioinformatics and genomics research because genetic studies require modelling of hundreds of thousands of genetic markers (i.e., many predictors; Strobl et al. 2007, 2008). These authors tend to focus on using random forests, exploring the desirable properties and limitations of this modelling approach.

		Bivariate Importance	
		Higher	**Lower**
Multivariate Importance	**Higher**	Variable is distinctly important	Interacts with other variable(s)
	Lower	Collinear with high-importance variable(s)	Variable is unimportant

Example Variable	Multivariate Importance	Bivariate Importance	Interpretation
X_1	7.4%	5.2%	High importance
X_2	8.9%	0.9%	Interaction
X_3	1.3%	4.8%	Collinear
X_4	1.2%	0.6%	Unimportant
X_5	4.0%	2.6%	Medium importance

Table 1.1 and 1.2. Multi-model interpretation matrix, along with example output. Conditional colour formatting implemented in Microsoft Excel.

A top *k* ranks approach is typically relied on for interaction detection, with the highest *k* importance variables being flagged for follow-up analysis (Wright et al. 2016). E.g., $k = 5$ means the top 5 most important variables are used in follow-up analysis. These authors then present a variety of modifications that enhance interaction detection. For example, Wright et al. (2016) and Bureau et al. (2005) discuss treating variables in pairs, so that importance scores correspond to the combined information provided by a given pair of variables. However, this approach can be computationally inefficient—considering every possible pair of predictors leads to an extensive expansion of the predictor-space when working with high-dimensional datasets. Wright et al. (2016) recommend selecting a subset of variable-pairs based on prior knowledge. However, we would like to develop a data exploration method that permits completely naïve detection of interactions (as well as other relationships).

Although many interaction detection researchers use ensemble methods, such as random forests, we have opted not to use this modelling approach because it is known to have issues with detecting interactions and collinearity. For example, Winham et al. (2012) showcase that random forests—particularly in high-dimensional contexts—capture main effects over interactions. Similarly, Toloşi et al. (2011) showcase that ensemble methods tend to dilute the importance of all correlated predictors. This is in contrast to non-ensemble methods, which tend to favour the more highly related multicollinear predictors and only shunt down the effects of lesser-related predictors (Kutner et al., 2005). For these reasons, we will explore the potential use of neural networks for interaction and collinearity detection. Although more tedious to implement, neural networks should maintain the desirable properties that we aim to leverage in our proposed method (Hornik 1991; Kutner et al. 2005; O'brien 2007).

Methods

Data generation process

We begin by specifying a data generation process to test the proposed method. This process involves generating 12 random variables X_p, sampled from a normal distribution with a mean of 0 and standard deviation of 1; $N(0,1)$:

$$X_1 \sim N(0,1),$$

$$X_2 \sim N(0,1),$$

…

$$X_{12} \sim N(0,1),$$

We then generate two covariates by creating columns that are equal to previously generated variables, plus random noise:

$$X_{13} \sim X_5 + N(0,1),$$

$$X_{14} \sim X_6 + N(0,1).$$

Finally, we generate an outcome variable Y with the following relationships to the various predictors X_p:

- Directly predicts Y with more strength than other predictors – X_1, X_2
- Directly predicts Y, interacts with noise – X_3, X_4
- Directly predicts Y, has covariate – X_5, X_6
- Directly predicts Y – X_7, X_8
- No direct relation to Y, but relates to Y via an interaction – X_9, X_{10}
- No relation to Y – X_{11}, X_{12}
- Covariate, no direct relation to Y – X_{13}, X_{14}

Two variables were generated for each relationship so that there is an alternative point-of-reference when judging whether importance metrics appear appropriately situated. Coefficients for most main effects were set to have a 0.5 effect size. The only exceptions being the first two predictors, whose effect sizes were set to 1.0. All interactions were set to have a 0.5 effect size. E.g., $Y = 1.0(X_1) - 1.0(X_2) + 0.5(X_3) - 0.5(X_4) \ldots + 0.5(X_4)(X_{10}) \ldots$. We additionally alternate the sign (+/-) of each effect intermittently, with odd-numbered predictors (e.g., X_1, X_3) having positive effects and even-numbered predictors (e.g., X_2, X_4) having negative effects. The alternating signs were implemented to ensure that only magnitude (and not direction) impact importance scores. This is expected given all importance scores in this study use either absolute or squared values for their calculations.

Although the modelling approaches used in this study are adept at handling non-normal data (Kutner et al. 2005), we opted to use normally distributed data for our initial tests of the stated method in order to observe its performance under ideal circumstances. Future research should vary the distributions being used in data generation. For a detailed summary of our simulation plan, see table 2. We start by generating n = 10,000 records from this data generation process, and then apply our proposed method to these data. This is done as a basic demonstration of how the method operates.

Next, we expand this initial data generation process into a Monte Carlo simulation. We take each unique relationship between predictors X_p and outcome Y generated above and experimentally vary their associated effect sizes. We do this by adding a 15th predictor to the simulation plan in table 2—we refer to this 15th variable as Xe_p (the "e" standing for "experimental"; thus Xe_p refers to an experimental predictor with the same relationship to Y as X_p, where p ranges from 1 to 14, representing each of the original predictors specified in the original data generation process).

Thus the new data generation equation is as such (note, this is identical to what is specified in table 2 except for the last term $\beta[Xr_p]$):

$$Y \sim 1.0(X_1) - 1.0(X_2) + 0.5(X_3) - 0.5(X_4) + 0.5(X_5) - 0.5(X_6) + 1.0(X_7) - 0.5(X_8) + 0.5(X_3)(X_9) - 0.5(X_4)(X_{10}) + N(0,1) + \beta(Xr_p), \quad (1)$$

where the effect size β experimentally varies from 0 to 1.25 in increments of .25 and Xr_p can be interchanged with any of the *relationships* expressed by X_1 through to X_{14}.

Given that there are two variables for each relationship (e.g., X_1 and X_2 have the same relationship with Y, only their sign [+/-] differs), we consider only odd-numbered predictors (e.g., X_1, X_3, X_5, etc.) as candidates for this experiment. We additionally exclude Xe_7, because it is redundant with Xe_1. Consider that X_1 and X_7 have only main effects—their only difference is their respective effect sizes of 1.0 and 0.5. If we systematically vary these effect sizes, an evaluation of Xe_1 will be identical to an evaluation of Xe_7.

Thus, throughout the simulation Xr_p is interchanged with:

$$Xr_1 = Xe_1,$$

$$Xr_3 = Xe_3 + Xe_3 * X_9,$$

$$Xr_5 = Xe_5;\ Xe_{13} \sim Xe_5 + N(0,1),$$
$$Xr_9 = X_3 * Xe_9,$$
$$Xr_{11} = 0.$$

Note that Xe_{13} is evaluated in parallel with Xe_5, because Xe_{13} is collinear with Xe_5 but has no independent relation to the outcome *Y*. Thus, only the effect size of Xe_5 changes, and we observe how this impacts the importance scores associated with experimental predictors Xe_5 and Xe_{13}.

Predictors X_p generation process			**Outcome *Y* generation process**
Relation to outcome *Y*	**Variable Name**	**Distribution**	**Term**
-	-	-	**Y =**
Direct Relationship	X_1	~ N(0,1)	+ 1.0*X_1
Direct Relationship	X_2	~ N(0,1)	- 1.0*X_2
Direct Relationship	X_3	~ N(0,1)	+ 0.5*X_3
Direct Relationship	X_4	~ N(0,1)	- 0.5*X_4
Direct Relationship	X_5	~ N(0,1)	+ 0.5*X_5
Direct Relationship	X_6	~ N(0,1)	- 0.5*X_6
Direct Relationship	X_7	~ N(0,1)	+ 0.5*X_7
Direct Relationship	X_8	~ N(0,1)	- 0.5*X_8
Interaction	X_9	~ N(0,1)	+ 0.5*X_3*X_9
Interaction	X_{10}	~ N(0,1)	- 0.5*X_4*X_{10}
Error term	-	-	+ N(0,1)
Noise	X_{11}	~ N(0,1)	
Noise	X_{12}	~ N(0,1)	
Covariate	X_{13}	~ X_5 + N(0,1)	
Covariate	X_{14}	~ X_6 + N(0,1)	

Table 2. Data Generation process for initial demonstration.

The first three columns on the left correspond to the generation of predictor variables X_p. The last column on the right specifies the relationship between each predictor X_p and the outcome variable *Y*.

For each simulation run, we start by generating *n* records with all X_p and Xe_p variables (*X1* through to X_{14}; Xe_1, Xe_3, Xe_5, Xe_9, Xe_{11}, Xe_{13}). These are all generated from a normal distribution with a mean of 0 and standard deviation of 1 (N[0,1]); except for Xe_{13}, which is equal to Xe_5 + N(0,1). For each relationship Xr_p at each level of effect size β, we generate an outcome *Y* using equation 1. Note that this results in 30 experimental conditions (5 Xr_p values × 6 β values = 30 conditions), with the *n* records being used once for every experimental condition. We then fit our bivariate and multivariate models in each experimental condition to derive importance scores for each Xe_p. These importance scores are standardized and appended to a dataset with information identifying their respective experimental condition. This entire process is then iterated over again for each subsequent simulation run, for a total of 120 simulation runs. This iterating is done so that variance in the performance of the presented method can be assessed and confidence intervals for importance scores can be drawn for each experimental condition. Additionally, we perform these 120 simulations for 2 different sample sizes (*n* = 500; 4,000), for a total of 240 simulation runs.

All data were generated using R, version 3.4.1 (R Core Team 2017). See the supplementary materials for pertaining code.

Models

LOESS Regression

For our bivariate models, we use Cleveland's (1979, 1993; Cleveland and Devlin 1988) LOESS method, taking the variance accounted for by the model (i.e., R-squared; Kutner et al. 2005) as our importance metric, calculated as:

$$R^2 = 1 - (\Sigma[Y_i - \hat{Y}_i]^2 / \Sigma[Y_i - \bar{Y}]^2). \quad (2)$$

Here, Y_i represents the *Y* value observed for the *i*th observation, where each of *n* observations is counted as $i = 1, i = 2, \ldots i = n$. $\hat{Y}_i$ represents the predicted value from the LOESS model for the *i*th

observation, whereas $\bar{Y}$ represents the grand mean of *Y*. Accordingly, $Y_i - \hat{Y}_i$ represents the model error for each observation, whereas $Y_i - \bar{Y}$ represents how far each observation falls from the mean of *Y*. Our numerator is the sum of squared error from our model, whilst our denominator represents the un-modelled sum of squared error in *Y*. Therefore, by taking the compliment of this quotient we obtain the proportion of squared error in *Y* accounted for by the model.

Detailed descriptions of LOESS regression can be found in other texts (Cleveland 1979; Kutner et al. 2005). Briefly, the LOESS method involves building a unique regression for every single data-point in the dataset, using the nearest *f*-percent subset of observations (where *f* is a percentage determined by the analyst). Typically, the *f*-percent nearest data-points are determined using Euclidian distance within the *X*-space (predictor-space). Within each regression, the contributing data-points are subsequently weighted by this distance (or a function thereof). Then, each fitted value is linked together by drawing a direct line (plane, or hyperplane) to the next nearest fitted value.

LOESS regression is extremely robust within low-dimensional contexts and can be used as an automatic fitting method (Cleveland 1979, 1993; Kutner et al. 2005). However, LOESS becomes increasingly computationally expensive as more predictors and data-points contribute to the model, as is typical with nearest-neighbour-type models. Determining Euclidian distance between data-points in higher dimensional space is more computationally expensive; each data-point needs its distance calculated between it and every other data-point, so more observations *n* means more computations are necessary. Ultimately, LOESS regression is ideal to use for our bivariate models but is unsuitable for our multivariate model.

LOESS models were built in R version 3.4.1 (R Core Team 2017), with a smoothing parameter of .4 (i.e., f = 40%). See the supplementary materials for pertinent code. There are formal methods for tuning the smoothing parameter (Cleveland and Devlin 1988). However, we tend to pick a value in the low-to-medium range (.2 to .6); using lower values in the presence of many data points (e.g., $n > 1{,}000{,}000$) and higher values in the presence of few data points (e.g., $n < 1{,}000$). This loose tuning approach aims to ensure that a sufficient number of points are used in calculating each local regression. We perform validation via visual checks of model fit and residuals. LOESS regression is typically used for visually exploring data (Kutner et al. 2005), wherein the absolute-optimal model is not necessary. This is the philosophy with which we approach this method.

Neural Networks

For our multivariate approach we use a *single-hidden-layer feedforward neural network* (Kutner et al. 2005; Raschka and Mirjalili 2017; Rosenblatt 1958). The advantage of using a neural network here is that the model should be able to mimic the data generation process of our outcome *Y* accurately (Hornik 1991).

Detailed descriptions of neural networks can be found in other texts (Kutner et al. 2005; Raschka and Mirjalili 2017). Briefly, the single-hidden-layer feedforward neural network involves building *m* regressions (referred to as *hidden nodes*), wherein all predictors are used, and then aggregating the results from these regressions to obtain a final predicted value. Each hidden node is fitted in conjunction with every other hidden node during optimization—hidden nodes are not independent regressions; the entire neural network is a singular excessively-parameterized model.

For the initial demonstration using n = 10,000 records, we trained the neural network via 4-fold cross-validation and grid search, tuning:

- the number of nodes in our hidden-layer—we started with m = 5, 10, 15, 20 (10 was selected as the best tune; we then tested different values surrounding 10, with m ranging from 8 to 12); and
- the regularization parameter λ (where $\lambda = 10^{-a}$; with a ranging from 0.5 to 6.0, in increments of 0.5; Kutner et al 2005; Zell et al 1998).

Once the optimal hyper-parameters were obtained (m = 9; a = 2.5), we retrained the neural network using all data. Garson's (1991; Goh 1995) variable importance was calculated for each predictor X_p. This neural network was trained using resilient backpropagation (Zell et al. 1998) with weights randomly initialized between +0.3 and -0.3. For pertinent code, see the supplementary materials. This neural network was implemented using the Stuttgart Neural Network Simulator (Zell et al. 1998) R package (RSNNS; Bergmeir and Benitez 2012) using R version 3.4.1. The caret package (Kuhn 2008) was used for *k*-fold cross-validation, Garson's (1991) importance scores were calculated using the NeuralNetsTools package (Beck 2018).

Note that the neural network implementation in our Monte Carlo simulation is slightly different from

that in the initial demonstration; we grid searched m = 10, 13, 15 crossed with λ = .0001, .00005, .00001 for each simulation run using 2-fold cross-validation and ordinary backpropagation (as opposed to resilient backpropagation). These discrepancies are due to the latter implementation being easier to manipulate in a Monte Carlo setting.

Results

Results from our initial demonstration using the sample of n = 10,000 records are presented in table 3. We provide supplementary information on the data generation process to validate interpretation. Note that interaction effects are listed twice; once for the first variable involved in the interaction, and a second time for the second variable involved in the interaction. We list collinear relations twice in this manner as well. This was done to make it easy to spot which variables were involved in given relationships. For the actual data generating equation, the reader can refer back to table 2.

	Results			Outcome Y Generation Process		
Variable	**Multivariate Importance**	**Bivariate Importance**	**Interpretation**	**Main Effects**	**Interactions**	**Collinear Relations**
$\boldsymbol{X_1}$	7.28%	19.26%	High importance	+ 1.0*X_1		
$\boldsymbol{X_2}$	7.85%	19.88%	High importance	- 1.0*X_2		
$\boldsymbol{X_3}$	12.73%	5.44%	Interaction	+ 0.5*X_3	+ 0.5*$\boldsymbol{X_3}$*X_9	
$\boldsymbol{X_4}$	15.94%	5.37%	Interaction	- 0.5*X_4	- 0.5*$\boldsymbol{X_4}$*X_{10}	
$\boldsymbol{X_5}$	4.61%	4.97%	Medium importance	+ 0.5*X_5		X_{13} = $\boldsymbol{X_5}$ + N(0,1)
$\boldsymbol{X_6}$	4.33%	4.49%	Medium importance	- 0.5*X_6		X_{14} = $\boldsymbol{X_6}$ + N(0,1)
$\boldsymbol{X_7}$	3.80%	4.99%	Medium importance	+ 0.5*X_7		
$\boldsymbol{X_8}$	4.13%	5.39%	Medium importance	- 0.5*X_8		
$\boldsymbol{X_9}$	16.13%	0.04%	Interaction		+ 0.5*X_3*$\boldsymbol{X_9}$	
$\boldsymbol{X_{10}}$	14.48%	0.06%	Interaction		- 0.5*X_4*$\boldsymbol{X_{10}}$	
$\boldsymbol{X_{11}}$	2.88%	0.11%	Unimportant			
$\boldsymbol{X_{12}}$	1.67%	0.10%	Unimportant			
$\boldsymbol{X_{13}}$	1.29%	2.77%	Collinear			$\boldsymbol{X_{13}}$ = X_5 + N(0,1)
$\boldsymbol{X_{14}}$	2.90%	3.05%	Collinear			$\boldsymbol{X_{14}}$ = X_6 + N(0,1)

Table 3. Demonstration of the proposed method using n = 10,000 records.

Results are displayed alongside the outcome Y generation process for ease of reference.

Results from our Monte Carlo simulation are presented in figure 2. The two left-most columns showcase the standardized importance scores (vertical axis) for the multivariate (in red) and bivariate (in blue) models by increasing effect size (horizontal axis). These two columns are ordered, from left to right, by increasing sample size used (n = 500; 4,000) in each simulation run. Error bars drawn at each effect size level β represent 95% empirical confidence intervals, where the bottom 2.5 and top 97.5 percentiles of simulation results are used as the cut-offs for the intervals.

Overlap in intervals between multivariate importance scores make it hard to differentiate when the multivariate score truly falls above or below the bivariate score. To address this, the right-most column shows the difference between standardized multivariate and bivariate scores at each effect size β for results where n = 4,000 (*difference = standardized multivariate importance – standardized bivariate importance*). Similarly, 95% empirical confidence intervals are drawn for these differences between the importance scores.

Discussion

Overall, most results are in line with expectation, with some caveats to discuss. We found the lower-sample size (n = 500) run not to be effective, with the method only performing as-expected for a few variables at certain effect size cut-offs (e.g., for Xe_9 when β >= 0.75). Therefore, the findings below are reported for each experimental variable Xe_p when a sample size of 4,000 is used. In practice, the lowest sample size dataset we wish to implement this method on is one of approximately 40,000 records. Thus,

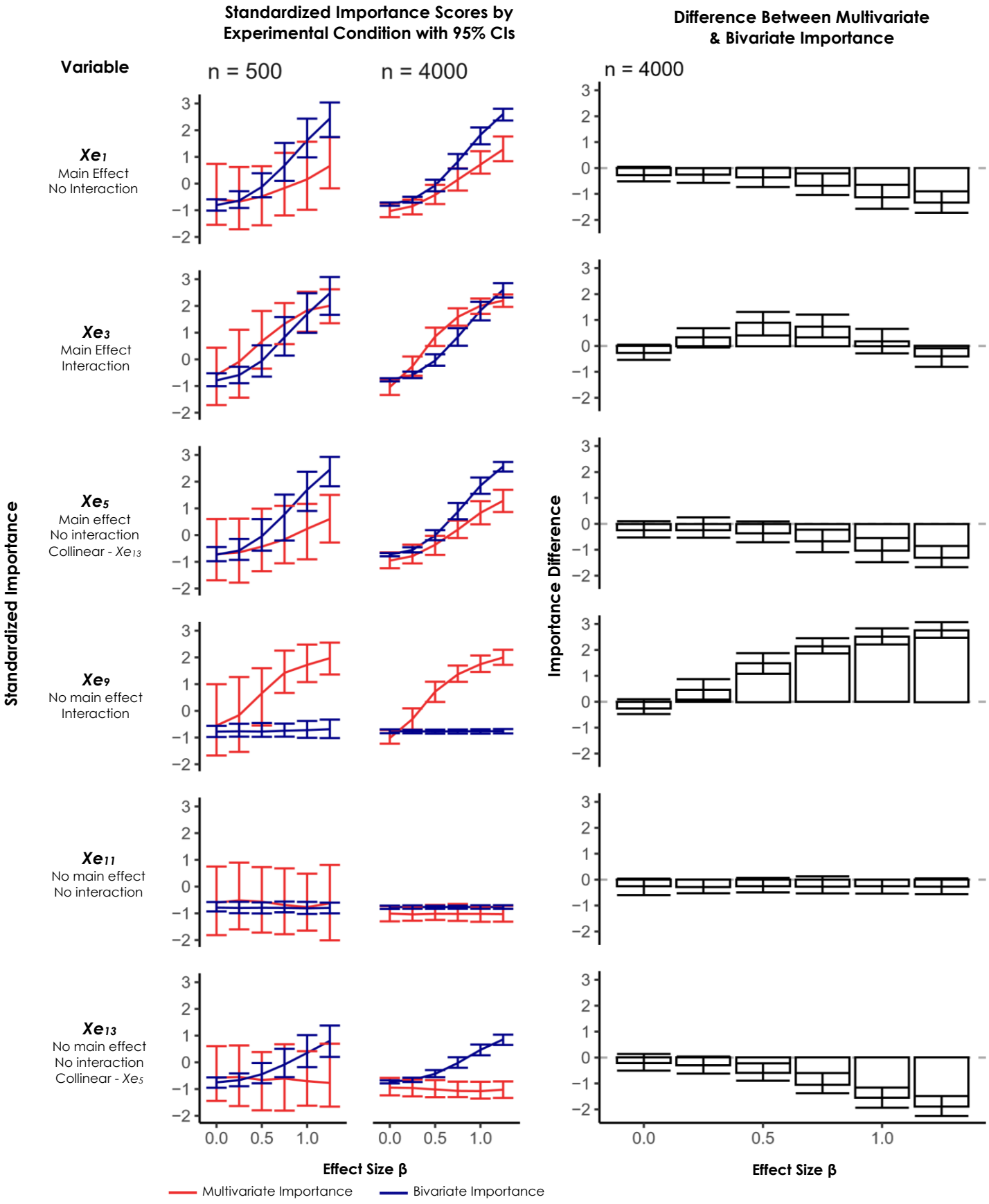

Figure 2. Standardized importance scores by experimentally varied effect sizes, for each experimental variable and sample size. Right-most column showcases differences between standardized multivariate and bivariate importance scores by varied effect size, internal to each simulation run. Figure created in R. CIs: Confidence Intervals.

we are most interested in the performance of our method among higher sample sizes.

Results for Xe_1 (main effect; no interaction) are perfectly in line with expectation. Both bivariate and multivariate importance scores increase uniformly with increasing effect size β.

Results for *Xe3* (main effect; interaction) are roughly in line with expectation. Of interest, the expected multivariate and bivariate importance scores for an interacting variable are most clearly observed when the effect size β ranges from 0.25 to 0.75. These are the effect sizes at which *Xe3* is most distinguishable from *Xe1*. It becomes noticeably harder to differentiate *Xe3* from *Xe1* once the effect size β for *Xe3* extends to 1.00 and above. Note that effect sizes β >= 1.00 are on the highest end of those used in the data generation equation, meaning we expect their standardized importance scores to be relatively high. Within our current implementation, it appears that when a variable has both a main effect and interaction with relatively high effect sizes, the main effect overshadows the interaction, making the interaction harder to detect. Using the right-most column in figure 1, we see that when β = 0.50 and 0.75, our 95% confidence intervals always fall above 0; meaning the multivariate importance overshoots the bivariate importance in the vast majority of simulation runs. However, this is not the case when β >= 1.00, indicating the interaction will go undetected by our method some of the time in these higher effect sizes. For β = 0.25, the confidence interval from the right-most column indicates that only a few simulation runs within our 95% confidence interval fall below 0, meaning interactions should be distinguishable at this effect size the majority of the time.

Results for Xe_5 (main effect; no interaction; collinear with Xe_{13}) are perfectly in line with expectation. Results appear almost identical to those of Xe_1. Most importantly, the multivariate model appears to maximize its usage of Xe_5 and neglect Xe_{13}, which is the desired result.

Results for Xe_9 (no main effect; interaction) are also perfectly in line with expectation. The bivariate importance flatlines at low importance across all effect size levels, whereas the multivariate importance increases with increasing effect size; the bivariate model does not pick up on the interaction, whereas the multivariate model does. This result strongly indicates that Xe_9 interacts with some other predictor. Similar to Xe_3, there may be slight ambiguity for the low effect size of β = 0.25, though the right-most column again shows that the majority of the time our importance differences fall above 0 for this effect size, making the interaction effect distinguishable more often than not.

Results for Xe_{11} (no main effect; no interaction) are perfectly in line with expectation. Xe_{11} has no relation to the outcome *Y* and, thus, flatlines, with low importance across all effect size levels.

Results for Xe_{13} (no main effect; no interaction; collinear with Xe_5) are in line with expectation. Results appear as approximate opposites to those observed for Xe_9; the multivariate importance flatlines at low importance across all effect size levels, whereas the bivariate importance increases with increasing effect size. Here, the multivariate model does not pick up on the collinear predictor, whereas the bivariate model does. This result strongly indicates that Xe_{13} is collinear with some other useful predictor. The only exception is for effect size β = 0.25—at this level it appears difficult to distinguish Xe_{13} from other variables (e.g., Xe_1, Xe_5).

Overall, our findings are as expected. However, some minor issues with Xe_3 should be taken into consideration when implementing the proposed method. Additionally, the method demonstrated reduced performance for some variables when dealing with low effect sizes (in particular when β = 0.25).

Generally, results show that the proposed multi-model method has promise; with main effects, interactions, collinearity, and unimportant variables being correctly classifiable based on outputs. An exception is that variables with both main effects and interactions may have their interactions missed when the size of their main effects are large relative to the effects of other variables in the dataset. Another exception is that performance is somewhat reduced (but not necessarily inadequate) for certain variables when their effect sizes are relatively low.

Limitations and future direction

While results from this study appear promising, the showcased tests of the proposed method are contextually limited. For example, there were no tests of three-variable (or more) interactions, categorical variables, varied degrees of collinearity between variables, or varied degrees of noise/error in the data generation process. Additionally, we would like this method to be applicable for large-scale data exploration problems, and thus will aim to test this method on datasets with 1,000s or 10,000s of variables to determine the feasibility of implementation. Likewise, the distributions used for data generation should be

permitted to vary, so that the present approach can be appraised under less ideal circumstances. These tests should be conducted in both simulated and applied contexts.

We are also curious about whether the method under study could be used as an alternative in other applied settings. For instance, might this method be useful as an alternative to step-wise selection during naïve model building? Additionally, could this method be effective for interaction detection within the bioinformatics context? At face value, it appears as though our approach could contend with other methods discussed in the literature (e.g., Wright et al. 2016), but direct tests may reveal unforeseen issues.

Although there is still much work to be done in validating the proposed method, it appears that the high-level idea works as intended. By combining complimentary information from (a) an overall multivariate model and (b) a series of bivariate models, one can make inference on the type of relationship existing between a given predictor X_p and outcome Y. Whilst more efficient or performant operationalisations of this idea may surface in future, the idea itself appears valid.

Instructions for Application

If interested in applying this method, we recommend sorting variables by their multivariate importance (in descending order). An analyst can then go through and label each variable, using the interpretation matrix presented in table 1.1. Although this may sound tedious, large groupings of variables are often categorizable in bulk (e.g., unimportant variables tend to cluster together at the bottom of the output and can be easily mass-categorized). This step usually takes no more than thirty minutes, even when looking at thousands of variables. Output for this is shown in table 4. These are completely fictional data, which we have written out for demonstrative purposes. Readers may borrow further inspiration on how to apply this method from the bioinformatics literature (e.g., Wright et al. 2016; Xu et al. 2020).

Variable Name	Multivariate Importance	Bivariate Importance	Interpretation
FAKEVAR80	10.19%	1.86%	Interaction
FAKEVAR290	10.03%	31.01%	High importance
FAKEVAR486	9.94%	31.51%	High importance
FAKEVAR100	7.65%	20.30%	High-medium Importance
FAKEVAR971	7.27%	23.05%	High-medium Importance
FAKEVAR837	6.57%	21.12%	High-medium Importance
FAKEVAR440	5.33%	0.05%	Interaction
FAKEVAR715	4.96%	0.56%	Interaction
FAKEVAR176	4.72%	14.71%	Medium Importance
FAKEVAR533	4.54%	16.06%	Medium Importance
FAKEVAR499	4.46%	14.83%	Medium Importance
FAKEVAR343	0.9%	0.2%	Unimportant
FAKEVAR991	0.6%	2.5%	Unimportant
FAKEVAR790	0.5%	3.7%	Unimportant
FAKEVAR789	0.5%	1.9%	Unimportant
FAKEVAR974	0.4%	2.9%	Unimportant
FAKEVAR728	0.2%	13.0%	Colinear
FAKEVAR484	0.2%	2.8%	Unimportant
FAKEVAR546	0.1%	16.1%	Colinear
…	…	…	…

Table 4. Output for fictional applied example.

Conclusion

In sum, we presented a multi-model method for classifying predictors based on their relationship to an outcome variable. We demonstrated the proposed method using simulated data. In addition, we applied our method to data with experimentally-varied effect sizes between each kind of relationship with the outcome variable to see how well the method could detect various relationships of differing strengths. Our findings indicate that the proposed method works as intended at varying effect size levels. However, although promising, there are many avenues to further validate the proposed method. We Additionally provide some instructions for application if readers wish to implement the proposed method.

References

Beck MW (2018) NeuralNetTools: Visualization and analysis tools for neural networks. J Stat Softw. http://doi.org/10.18637/jss.v085.i11

Bergmeir C, Benitez JM (2012) Neural networks in R using the Stuttgart neural network simulator: RSNNS. J Stat Softw. https://doi.org/10.18637/jss.v046.i07

Bureau A, Dupuis J, Falls K, Lunetta KL, Hayward B, Keith TP, Van Eerdewegh P (2005) Identifying SNPs predictive of phenotype using random forests. Genet Epidemiol. https://doi.org/10.1002/gepi.20041

Cleveland WS (1979) Robust locally weighted regression and smoothing scatterplots. J Am Stat Assoc. https://doi.org/10.1080/01621459.1979.10481038

Cleveland WS (1993) Visualizing Data. Hobart Press, Chicago.

Cleveland WS, Devlin SJ (1988) Locally weighted regression: An approach to regression analysis by local fitting. J Am Stat Assoc. https://doi.org/10.2307/2289282

Fisher RA (1936) The use of multiple measurements in taxonomic problems. Annual Eugenics. https://doi.org/10.1111/j.1469-1809.1936.tb02137.x

Garson GD (1991) Interpreting neural network connection weights. Artificial Intelligence Expert 6:46-51.

Goh ATC (1995) Back-propagation neural networks for modeling complex systems. Artificial Intelligence in Engineering. https://doi.org/10.1016/0954-1810(94)00011-S

Hornik K (1991) Approximation capabilities of multilayer feedforward networks. Neural Netw. https://doi.org/10.1016/0893-6080(91)90009-T

Kuhn M (2008) Caret package. J Stat Softw. https://doi.org/10.18637/jss.v028.i05

Kutner MH, Nachtsheim CJ, Neter J, Li W (2005) Applied linear statistical models (5th ed.). McGraw-Hill/Irwin, New York.

O'brien RM (2007) A caution regarding rules of thumb for variance inflation factors. Qual Quant. https://doi.org/10.1007/s11135-006-9018-6

Schloerke B, Cook D, Larmarange J, Briatte F, Marbach M, Thoen E, Elberg A, Toomet O, Crowley J, Hofmann H, Wickham H (2018) GGally: Extension to 'ggplot2'. R package version 1.4.0. https://CRAN.R-project.org/package=GGally

Strobl C, Boulesteix AL, Kneib T, Augustin T, Zeileis A (2008) Conditional variable importance for random forests. BMC Bioinform. https://doi.org/10.1186/1471-2105-9-307

Strobl C, Boulesteix AL, Zeileis A, Hothorn T (2007) Bias in random forest variable importance measures: Illustrations, sources and a solution. BMC Bioinform. https://doi.org/10.1186/1471-2105-8-25

R Core Team (2017) R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria. https://www.R-project.org/ Accessed 06 December 2022

Raschka S, Mirjalili V (2017) Python machine learning. Pakt, Birmingham.

Rosenblatt F (1958) The perceptron: A probabilistic model for information storage and organization in the brain. Psychol Rev. https://doi.org/10.1037/h0042519

Toloşi L, Lengauer T (2011) Classification with correlated features: unreliability of feature ranking and solutions. Bioinformatics. https://doi.org/10.1093/bioinformatics/btr300

Winham SJ, Colby CL, Freimuth RR, Wang X, de Andrade M, Huebner M, Biernacka JM (2012) SNP interaction detection with Random Forests in high-dimensional genetic data. BMC Bioinform.

https://doi.org/10.1186/1471-2105-13-164

Wright MN, Ziegler A, König IR (2016) Do little interactions get lost in dark random forests? BMC Bioinform. https://doi.org/10.1186/s12859-016-0995-8

Xu Y, Cao L, Zhao X, Yao Y, Lui Q, Zhang B, Wang Y, Mao Y, Ma Y, Ma JZ, Payne TJ, Li M D, Li L. (2020) Prediction of smoking behavior from single nucleotide polymorphisms with machine learning approaches. Front Psychiatry. https://doi.org/10.3389/fpsyt.2020.00416

Zell A, Mamier G, Vogt M, Nache N, Hübner R, Döring S, Herrmann KU, Soyez T, Schmalzl M, Sommer T, Hatzigeorgiou A, Posselt D, Schreiner T, Kett B, Glemente G, Wieland J, Gatter J, Reczko M, Riedmiller M, Seemann M, Ritt M, DeCoster J, Biedermann J, Danz J, Wehrfritz C, Werner R, Berthold M, Orsier B (1998) SNNS Stuttgart Neural Network Simulator User Manual, Version 4.2. IPVR, University of Stuttgart and WSI, University of Tubingen. http://www.ra.cs.uni-tuebingen.de/SNNS/